CONSOLIDATED SOFTWARE DESIGN DOCUMENT

Notification Service

Mini E-Commerce Platform

Reconciled from 4 source design documents into a single authoritative reference

Document Type	Consolidated Software Design Document (SDD / HLD)
Component	Notification Service (Microservice)
System	Mini E-Commerce Platform (Product Service, Order Service, Notification Service)
Version	2.0 (supersedes v1.0 and all individually-sourced drafts)
Status	Draft for Tech Lead Review
Date	July 16, 2026
Target Implementation Stack	Java 17, Spring Boot 4.1.0, Spring Data JPA, PostgreSQL, Maven, Lombok

Revision History

Version	Date	Author	Description
1.0	2026-07-16	Notification Service Engineering	Initial simulated-logging-only draft (single source)
2.0	2026-07-16	Solution Architecture	Consolidated from HLD.pdf, BRD/Technical Design, Design Document, and v1.0 into one reconciled reference — see §0 for every resolved conflict

Prepared for the Mini E-Commerce Platform engineering organization

Table of Contents

0. Document Provenance & Reconciliation Notes

#	Source Document	Character
1	High-Level Design Document (HLD.pdf)	Most implementation-grounded — written after inspecting the actual sparta-notification-service repository (real package names, real pom.xml, Spring Boot 4.1.0 confirmed on disk). Includes full Java code, folder structure, task estimates, PR checklist.
2	Notification Service — BRD & Technical Design (notification-service-brd-design.pdf)	The most production-grade architecture — models the database as a durable job queue (SELECT ... FOR UPDATE SKIP LOCKED), with exponential backoff + jitter, a stuck-worker reaper, and a separate per-attempt audit table.
3	Notification Service — Design Document (Notification-Service-Design-Document.pdf)	A general-scenario design for a 5-developer team, organized around parallel work tracks and a 5-day sprint. Introduces a referenceId idempotency key, a PUSH channel, and design-pattern callouts.
4	Software Design Document (NOTIFI~1.pdf)	An earlier simulated-logging-only design produced in this same working session, covering the same ground at a lighter technical depth.

The four documents do not agree with each other on several load-bearing decisions. Rather than silently pick one or mechanically concatenate all four, each conflict below was resolved deliberately, as a senior architect reconciling competing designs would. Each decision is called out explicitly so it can be challenged in review.

Decision Point	Resolution Adopted	Rationale
Primary key type	UUID (not HLD's Long/BIGSERIAL)	Two of three architecture-level sources converged on UUID independently; it avoids exposing sequential volume and suits externally-sourced idempotency keys and future Kafka event IDs.
Lifecycle states	5-state model: PENDING → SENDING → SENT, with RETRYING and terminal FAILED (BRD's model)	HLD's 3-state model cannot distinguish “a worker is actively attempting delivery” from “nothing is happening” — if a worker crashes mid-send, that row is indistinguishable from a fresh one. SENDING is what makes crash recovery possible; this is a reliability gap, not a style difference.
Idempotency	Adopted — idempotency_key column, unique constraint, INSERT ... ON CONFLICT DO NOTHING	HLD has no idempotency key anywhere in its entity, DTOs, or schema, despite its own Section 32 assuming one will exist for future Kafka redelivery. Order Service (or Kafka later) will redeliver the same event more than once; without dedup, customers receive duplicate emails/SMS.
Retry engine	Phased: MVP ships HLD/Design-Document-level simplicity; a hardening phase adopts BRD's claim-queue design	BRD's design is the most correct architecture for the stated NFRs, but is more than a 5-day/5-developer or 12-day/1-developer delivery needs on day one. Phasing avoids both under-engineering (a real gap

Decision Point	Resolution Adopted	Rationale
		under concurrent instances) and over-engineering (a distributed job-queue before it's needed).
Audit trail	Adopted as a recommended (not day-one-mandatory) addition	Per-attempt history is cheap to add now and expensive to retrofit — existing rows have no history to backfill later.
Channels	Email/SMS in scope now; Push named as a near-zero-cost extension point	Two of three sources scope Push out for this phase, and the NotificationSender strategy interface already accommodates it without core changes.
Spring Boot version	4.1.0, per HLD's direct pom.xml inspection	An inspected fact from the real repository beats an assumption made without looking. Flagged as an open item for Tech Lead confirmation — Boot 4 renamed several starters relative to 3.x.
Security	API key header required now (Design Document's position), with HLD's OAuth2/mTLS upgrade path retained	A shared-secret header is near-zero cost and meaningfully reduces blast radius; no good reason to defer it to “a later phase.”
Team / delivery model	Both kept — the 5-track model (Design Document) as the primary staffing plan, with HLD's granular task list distributed into the appropriate track	Not actually a conflict once recognized as such — the same backlog under two different staffing assumptions.

Where the four documents agreed — layered Controller → Service → Repository architecture, PostgreSQL as the datastore, REST now / Kafka later, simulated Email/SMS dispatch via a swappable sender abstraction, structured logging with correlation IDs — this document adopts the shared position without further discussion.

1. Overview

1.1 Purpose

The Notification Service is a standalone microservice in the Mini E-Commerce Platform (alongside Product Service and Order Service) responsible for reliably informing customers when an order-related event occurs. This document specifies its requirements, architecture, data model, API contract, lifecycle, and delivery plan in enough detail to implement it in Java / Spring Boot against the real sparta-notification-service repository.

1.2 Scope

In scope (current phase):

- REST API to receive notification requests (initially from Order Service only).
- Durable persistence of every notification and its lifecycle status in PostgreSQL.
- Simulated Email/SMS dispatch via a swappable NotificationSender abstraction (no real provider integration yet).
- Idempotent request handling via a caller-supplied idempotency key.
- Automatic retry of transient failures, bounded by a configurable maximum, with a documented path to exponential backoff.
- Query APIs for notification history and status, by recipient, order, and status.
- Structured, correlated logging and centralized exception handling.

Out of scope (this phase):

- Real integration with email/SMS providers (SES, SendGrid, Twilio, MSG91) — stubbed via the sender abstraction.
- Kafka / event-driven ingestion — REST only for now; the design anticipates the swap (§5.4, §15).
- Push notifications — the channel abstraction supports adding it, but it is not built now.
- User notification preferences (opt-in/opt-out, quiet hours, channel preference).
- Message templating engines — the caller supplies a fully-rendered message.
- Multi-tenancy.

1.3 Responsibilities

The Notification Service is a leaf node: it does not call back into Order Service or Product Service, and it holds no product or order business logic. It knows only: an event happened, here is who to notify, here is what to say, here is how to send it.

1.4 Codebase & Environment Context

Before this document was written, the actual project workspace was inspected rather than assumed (carried forward from HLD.pdf's Section 0, since this is grounded fact rather than a design opinion that should be silently dropped):

- Three sibling repositories already exist as Spring Initializr skeletons at the same starting point — sparta-product-service, sparta-order-service, sparta-notification-service — each with its own independent Git history, confirming a strict database-per-service / repo-per-service boundary.
- The generated skeleton fixes groupId=com.training, artifactId=notification-service, base package com.training.notificationservice. This document uses that exact package rather than a generic placeholder.
- Spring Boot version discrepancy: the project's pom.xml parent is spring-boot-starter-parent version 4.1.0, not 3.x. This is reflected in starter artifact names already present (spring-boot-starter-webmvc, not -web; split -test starters). Action item: confirm with the Tech Lead whether 4.1.0 is intentional before implementation begins.
- Currently present dependencies: spring-boot-starter-data-jpa, spring-boot-starter-webmvc, postgresql (runtime), plus matching test starters.
- Missing dependencies required by this design: lombok, spring-boot-starter-validation. Recommended additions: springdoc-openapi-starter-webmvc-ui, flyway-core + flyway-database-postgresql, testcontainers + testcontainers-postgresql + testcontainers-junit-jupiter.
- application.properties currently only sets spring.application.name=notification-service — datasource, port, and JPA properties are not yet configured.
- No source code exists yet beyond the generated application class and its default test — this is a from-scratch build.
- sparta-order-service and sparta-product-service are at the identical bare-skeleton stage; neither has implemented the “Order Confirmed” publisher this service depends on. The contract in §8 is what must be handed to the Order Service owner before either side builds against it.

2. Functional Requirements

ID	Requirement
FR-1	The service shall expose a REST endpoint to accept a notification request containing an idempotency key, channel (EMAIL/SMS), recipient, and message (with optional subject).
FR-2	The service shall persist every accepted request immediately with status PENDING, before any dispatch is attempted — this is the durability checkpoint.
FR-3	Duplicate requests carrying an idempotency key already on file shall return the existing notification rather than creating a new one.
FR-4	The service shall attempt delivery asynchronously, decoupled from the create request's response.
FR-5	The service shall transition each notification through the defined lifecycle (§6) and persist every transition.
FR-6	On a transient failure (timeout, 5xx, HTTP 429), the service shall retry automatically up to a configured max_retries.
FR-7	On a permanent failure (e.g., malformed recipient), the service shall not retry and shall mark the notification FAILED immediately.
FR-8	When retries are exhausted, the service shall mark the notification FAILED (dead-letter) and retain it for inspection — never delete it.
FR-9	The service shall expose an endpoint to manually re-queue a FAILED notification.
FR-10	The service shall expose endpoints to fetch a notification by ID, by order ID, and to search/filter by recipient and status, with pagination.
FR-11	The service shall validate all inbound requests (required fields, valid enum values, recipient format matching channel) and return structured 400 errors on failure.
FR-12	The service shall record createdAt/updatedAt timestamps, and, where the audit table is implemented, one row per delivery attempt.

3. Non-Functional Requirements

Category	Requirement
Durability	A notification survives a process crash — it is persisted before acknowledgement, and no accepted request may be silently lost.
Delivery guarantee	At-least-once delivery, paired with idempotency so it behaves as effectively-once from the recipient's point of view.
Crash safety	A worker crashing mid-send leaves recoverable state (the SENDING status, §6); no notification is silently lost or stuck forever.
Scalability	Stateless at the application layer; horizontally scalable. Multiple instances must process retries concurrently without double-sending (Phase 2, §5.3).
Availability	No synchronous dependency on Product or Order Service at request time — only on its own database.
Performance	P99 API response time under 300 ms for write/query endpoints, excluding actual provider latency.
Observability	Every notification's lifecycle is traceable via structured logs and status history; failures carry a reason.
Extensibility	Adding a channel (Push) or swapping REST → Kafka ingestion does not touch the service or repository layers.
Maintainability	Layered architecture (Controller → Service → Repository), SOLID principles, DTOs isolate the persistence model from the wire contract.
Security	Input validation on all fields; service-to-service calls authenticated via a shared API key header, upgradeable to OAuth2/mTLS.
Testability	Business logic isolated in the service layer, independent of HTTP/DB, with a ≥80% coverage target.

4. Domain Model

4.1 Core Entity — Notification

Field	Type	Notes
id	UUID	Primary key (see §0 — reconciled from HLD's Long)
idempotencyKey	String	Caller-supplied, unique — the dedup key (e.g., order-1023-confirmed)
orderId	String	Logical reference to Order Service's order — no foreign key across service boundaries
channel	Enum	EMAIL, SMS
recipient	String	Email address or phone number, validated against channel
subject	String, nullable	Email only
message	String	Final rendered message body
status	Enum	PENDING, SENDING, RETRYING, SENT, FAILED (§6)
retryCount	Integer	Attempts made so far
maxRetries	Integer	Configurable, default 5
nextAttemptAt	Timestamp	When the row becomes eligible for the next attempt — data, not sleep(), so it survives restarts
lastError	String, nullable	Reason for the most recent failure
lockedAt	Timestamp, nullable	Set when a worker claims the row (Phase 2 reaper input)
createdAt	Timestamp	Request received time
updatedAt	Timestamp	Last status change
sentAt	Timestamp, nullable	Time of successful send

4.2 Supporting Entity — NotificationAttempt (audit trail, recommended)

Append-only; one row per delivery attempt. Keeps the hot notifications row lean while giving full history for free.

Field	Type	Notes
id	BIGINT identity	Primary key
notificationId	UUID	FK to notifications.id
attemptNumber	Integer	1, 2, 3...

Field	Type	Notes
outcome	Enum	SUCCESS, FAILURE
errorDetail	String, nullable	
attemptedAt	Timestamp	

4.3 Enumerations

```
public enum NotificationChannel { EMAIL, SMS }

public enum NotificationStatus { PENDING, SENDING, RETRYING, SENT, FAILED }
```

5. Architecture

5.1 Layered View

```
Client (Order Service / Support Tooling / Scheduler)
        |  REST (JSON over HTTP)
        v
  Controller Layer      <- REST contracts, DTOs, input validation
        v
  Service Layer         <- business rules, state machine, retry policy, mapping
        v
  Repository Layer  <--->  NotificationSender (SPI: Email / SMS, simulated)
        v
  PostgreSQL -- notifications (+ notification_attempts)
```

Key architectural decisions (agreed across all four source documents):

- Ports & Adapters for dispatch. NotificationSender is an interface; today it has simulated EmailNotificationSender/SmsNotificationSender implementations, tomorrow real SES/Twilio adapters — the service layer never changes.
- Write-ahead persistence. The row is saved as PENDING before a send is attempted, so a crash mid-send never loses the request.
- DTO isolation. Controllers never expose JPA entities; dedicated Request/Response DTOs decouple the wire contract from the persistence model.
- Stateless service. All state lives in PostgreSQL; any instance can serve any request.
- Database-per-service. No shared schema; orderId is a logical reference only, never a physical foreign key across the service boundary.

5.2 Architecture Diagram

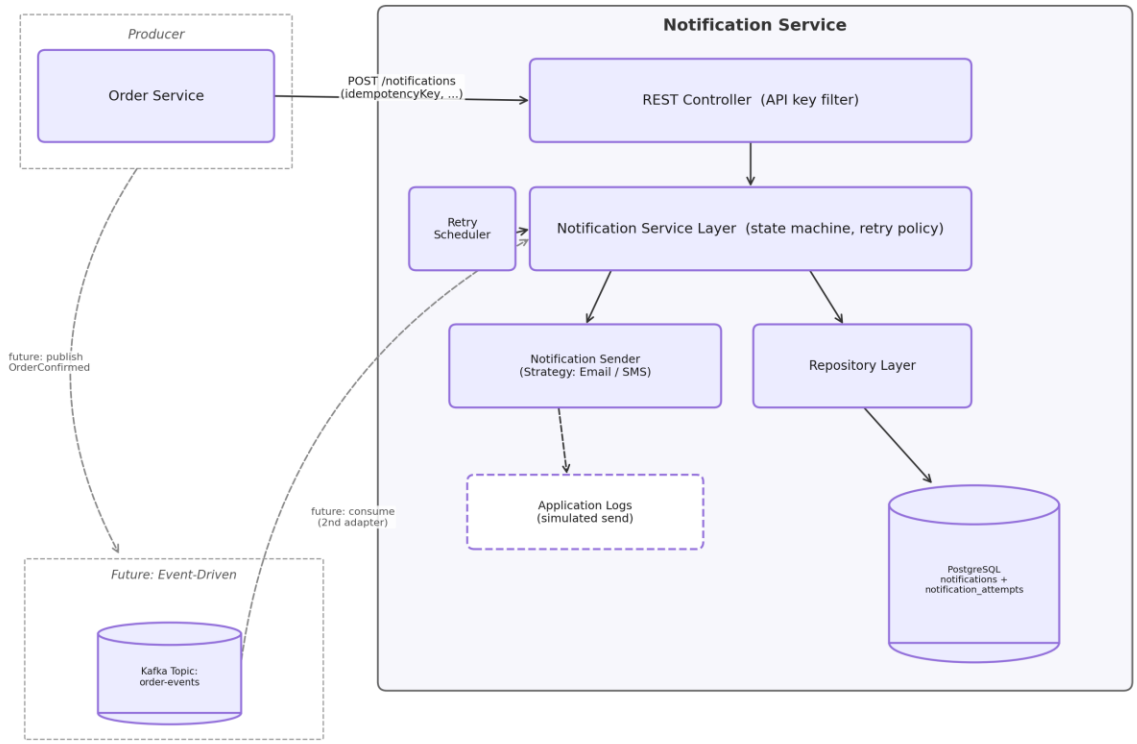

Figure 1 — Unified architecture: Order Service, layered Notification Service internals, and the future Kafka path.

5.3 Retry Engine — Phased Design

Reconciled per §0: ship the simpler design first, then harden.

Phase 1 (MVP — matches the delivery estimates in §13):

A single in-process @Scheduled poller runs every POLL_INTERVAL (e.g., 30s), selects RETRYING/PENDING rows whose nextAttemptAt <= now(), and re-invokes the sender. This is sufficient for a single running instance and satisfies FR-6–FR-9.

Phase 2 (hardening — adopt once horizontal scaling or real load arrives):

Replace the naive poller with an atomic claim query so N worker instances can run concurrently without double-sending:

```
WITH due AS (
    SELECT id FROM notifications
    WHERE status IN ('PENDING', 'RETRYING') AND next_attempt_at <= now()
    ORDER BY next_attempt_at
    LIMIT :batchSize
    FOR UPDATE SKIP LOCKED
)
UPDATE notifications n
SET status = 'SENDING', locked_at = now()
FROM due WHERE n.id = due.id
RETURNING n.*;
```

FOR UPDATE locks the selected rows; SKIP LOCKED tells concurrent workers to step over rows a peer already claimed. This single mechanism is what lets Postgres act as a durable, concurrent job queue with no additional broker infrastructure.

Phase 2 also adds a reaper — a scheduled job that resets rows stuck in SENDING past a timeout (e.g., 5 minutes) back to RETRYING, rescuing work orphaned by a crashed worker:

```
UPDATE notifications
SET status = 'RETRYING', next_attempt_at = now(), locked_at = NULL
WHERE status = 'SENDING' AND locked_at < now() - interval '5 minutes';
```

5.4 Future Integration with Kafka

Nothing in §6–§8 changes when Kafka arrives. A @KafkaListener-based OrderEventConsumer becomes a second ingestion adapter that maps an OrderConfirmed event to the same idempotent insert the REST controller performs today. Use the Kafka message/event ID as the idempotency key so redelivery is naturally deduplicated. The queue, worker, retry logic, channels, and schema are untouched — this is the payoff of separating ingestion from delivery, and is why the REST controller must stay thin and delegate everything to the service layer now.

6. Notification Lifecycle

Five states; SENT and FAILED are terminal.

State	Meaning	Terminal?
PENDING	Accepted and persisted; not yet attempted.	No
SENDING	A worker has claimed it and is attempting delivery — persisted, not in-memory.	No
RETRYING	A transient failure occurred; scheduled for a future attempt via nextAttemptAt.	No
SENT	Delivered successfully.	Yes
FAILED	Permanent failure or retries exhausted; retained as a dead-letter, never deleted.	Yes

Transitions:

- PENDING → SENDING: a worker (or the Phase-1 inline dispatch) claims the row and attempts delivery.
- SENDING → SENT: delivery succeeds.
- SENDING → FAILED: a permanent error occurs (e.g., malformed recipient) — retrying would be pointless.
- SENDING → RETRYING: a transient error occurs and budget remains; a future nextAttemptAt is scheduled.
- RETRYING → SENDING: the scheduled/claimed retry attempt begins.
- RETRYING → FAILED: retries are exhausted (retryCount >= maxRetries).

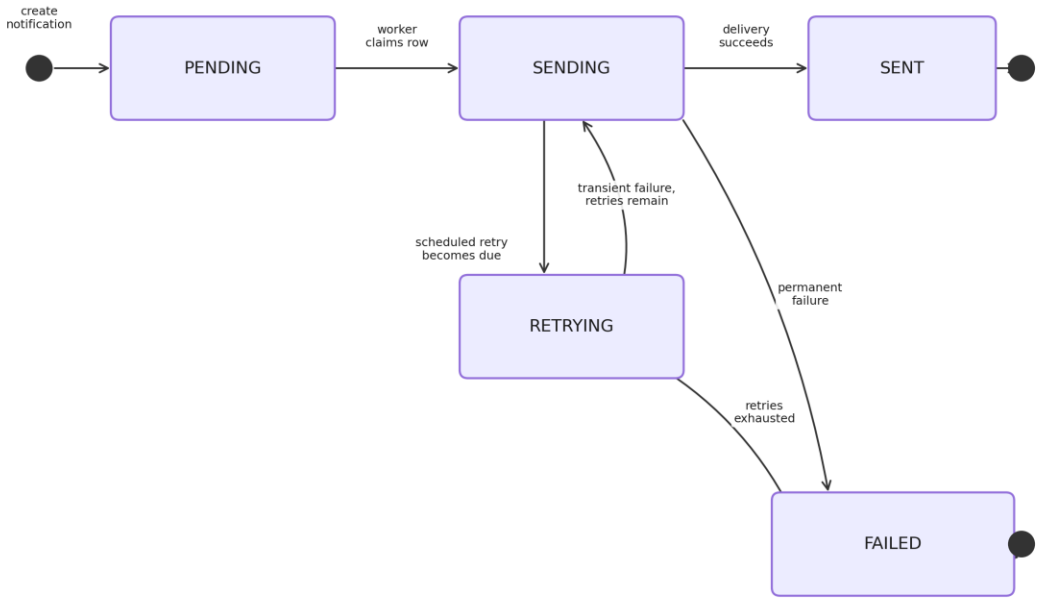

Figure 2 — Reconciled 5-state notification lifecycle.

7. Database Design

7.1 notifications (current state / the queue)

```
CREATE TABLE notifications (
    id                  UUID            PRIMARY KEY DEFAULT gen_random_uuid(),
    idempotency_key     TEXT            NOT NULL,
    order_id            TEXT            NOT NULL,
    channel             TEXT            NOT NULL CHECK (channel IN ('EMAIL', 'SMS')),
    recipient           TEXT            NOT NULL,
    subject             TEXT,
    message             TEXT            NOT NULL,
    status              TEXT            NOT NULL DEFAULT 'PENDING'
                         CHECK (status IN ('PENDING','SENDING','RETRYING','SENT','FAILED')),
    retry_count         INT             NOT NULL DEFAULT 0,
    max_retries         INT             NOT NULL DEFAULT 5,
    next_attempt_at     TIMESTAMPTZ     NOT NULL DEFAULT now(),
    last_error          TEXT,
    locked_at           TIMESTAMPTZ,
    created_at          TIMESTAMPTZ     NOT NULL DEFAULT now(),
    updated_at          TIMESTAMPTZ     NOT NULL DEFAULT now(),
    sent_at             TIMESTAMPTZ,

    CONSTRAINT uq_notifications_idempotency UNIQUE (idempotency_key),
    CONSTRAINT ck_notifications_retry_bounds CHECK (retry_count >= 0 AND retry_count <=
max_retries)
);

-- Hot path: worker scans only for due, pickable rows. Partial index excludes terminal rows.
CREATE INDEX idx_notifications_dispatch ON notifications (next_attempt_at)
    WHERE status IN ('PENDING', 'RETRYING');

-- Phase 2 reaper input
CREATE INDEX idx_notifications_locked ON notifications (locked_at) WHERE status = 'SENDING';

CREATE INDEX idx_notifications_order_id   ON notifications (order_id);
CREATE INDEX idx_notifications_status     ON notifications (status);
CREATE INDEX idx_notifications_recipient  ON notifications (recipient);
```

7.2 notification_attempts (append-only audit trail — recommended)

```
CREATE TABLE notification_attempts (
    id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    notification_id   UUID   NOT NULL REFERENCES notifications(id) ON DELETE CASCADE,
    attempt_number    INT    NOT NULL,
    outcome           TEXT   NOT NULL CHECK (outcome IN ('SUCCESS','FAILURE')),
    error_detail      TEXT,
    attempted_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_attempts_notification ON notification_attempts (notification_id);
```

Figure 3 — Entity-relationship diagram: notifications and notification_attempts.

7.3 Idempotent Insert Pattern

```
INSERT INTO notifications (idempotency_key, order_id, channel, recipient, subject, message,
max_retries)
VALUES ($1, $2, $3, $4, $5, $6, $7)
ON CONFLICT (idempotency_key) DO NOTHING
RETURNING id, status;
```

If a row is returned, a new notification was created. If nothing is returned, a duplicate request was received — fetch and return the existing row by idempotency_key. Either way, the create endpoint is idempotent. Deduplication is enforced by a database constraint, not a check-then-insert race in application code.

7.4 Three Deliberate Postgres Decisions

- status/channel as CHECK-constrained TEXT, not native ENUM. Postgres enum types are painful to evolve (adding a value means altering the type); a CHECK constraint evolves with a plain migration.
- UNIQUE (idempotency_key) enforced by the database, not application logic — a check-then-insert races under concurrency; a unique constraint with ON CONFLICT does not.
- Partial index on (next_attempt_at) WHERE status IN ('PENDING','RETRYING'). The dispatcher never cares about terminal rows, so they never enter the index — it stays small and fast even as SENT rows accumulate into the millions.

ddl-auto=validate is recommended over update/create; schema ownership belongs to a versioned Flyway migration under src/main/resources/db/migration/, not Hibernate auto-DDL.

8. API Design

Base path: /api/v1/notifications

#	Method	Endpoint	Purpose	Success
1	POST	/api/v1/notifications	Create/enqueue a notification (idempotent on idempotencyKey)	202 Accepted
2	GET	/api/v1/notifications/{id}	Fetch a single notification by ID	200 OK
3	GET	/api/v1/notifications/order/{orderId}	Fetch all notifications for an order	200 OK
4	GET	/api/v1/notifications?recipient=&status=&page=&size=	Search/filter with pagination	200 OK
5	POST	/api/v1/notifications/{id}/retry	Manually re-queue a FAILED notification	202 Accepted
6	GET	/api/v1/notifications/{id}/status	Lightweight status check	200 OK
7	GET	/actuator/health	Health check	200 OK

8.1 Create Notification

Request

```
POST /api/v1/notifications
Content-Type: application/json

{
  "idempotencyKey": "order-1023-confirmed",
  "orderId": "1023",
  "channel": "EMAIL",
  "recipient": "jane.doe@example.com",
  "subject": "Your order is confirmed",
  "message": "Hi Jane, your order #1023 has been confirmed."
}
```

Response — 202 Accepted

```
{
  "id": "8f14e45f-ceea-4f6e-9b2f-1a2b3c4d5e6f",
  "orderId": "1023",
  "channel": "EMAIL",
  "recipient": "jane.doe@example.com",
  "status": "PENDING",
  "retryCount": 0,
  "createdAt": "2026-07-16T10:15:30Z",
  "updatedAt": "2026-07-16T10:15:30Z"
}
```

8.2 Retry Notification

```
POST /api/v1/notifications/8f14e45f-ceea-4f6e-9b2f-1a2b3c4d5e6f/retry
```

Response — 202 Accepted

```
{
  "id": "8f14e45f-ceea-4f6e-9b2f-1a2b3c4d5e6f",
  "status": "RETRYING",
  "retryCount": 2,
  "updatedAt": "2026-07-16T10:20:00Z"
}
```

8.3 Error Response (all endpoints)

```
{
  "timestamp": "2026-07-16T10:22:11Z",
  "status": 404,
  "error": "NOT_FOUND",
  "message": "Notification not found with id: 999",
  "path": "/api/v1/notifications/999"
}
```

8.4 HTTP Status Codes

Code	Meaning	Used By
200 OK	Successful read	GET endpoints
202 Accepted	Accepted for asynchronous processing	POST create, POST retry
400 Bad Request	Validation failure	All endpoints
404 Not Found	Resource does not exist	GET by id, POST retry
409 Conflict	Not in a retryable state (e.g., already SENT)	POST retry
500 Internal Server Error	Unexpected server error	All endpoints

8.5 Request Lifecycle — Sequence Diagram

The diagram below traces a single notification end-to-end: Order Service submits the request, the idempotent insert runs, the caller is acknowledged immediately, and the worker attempts delivery asynchronously, updating status based on the outcome.

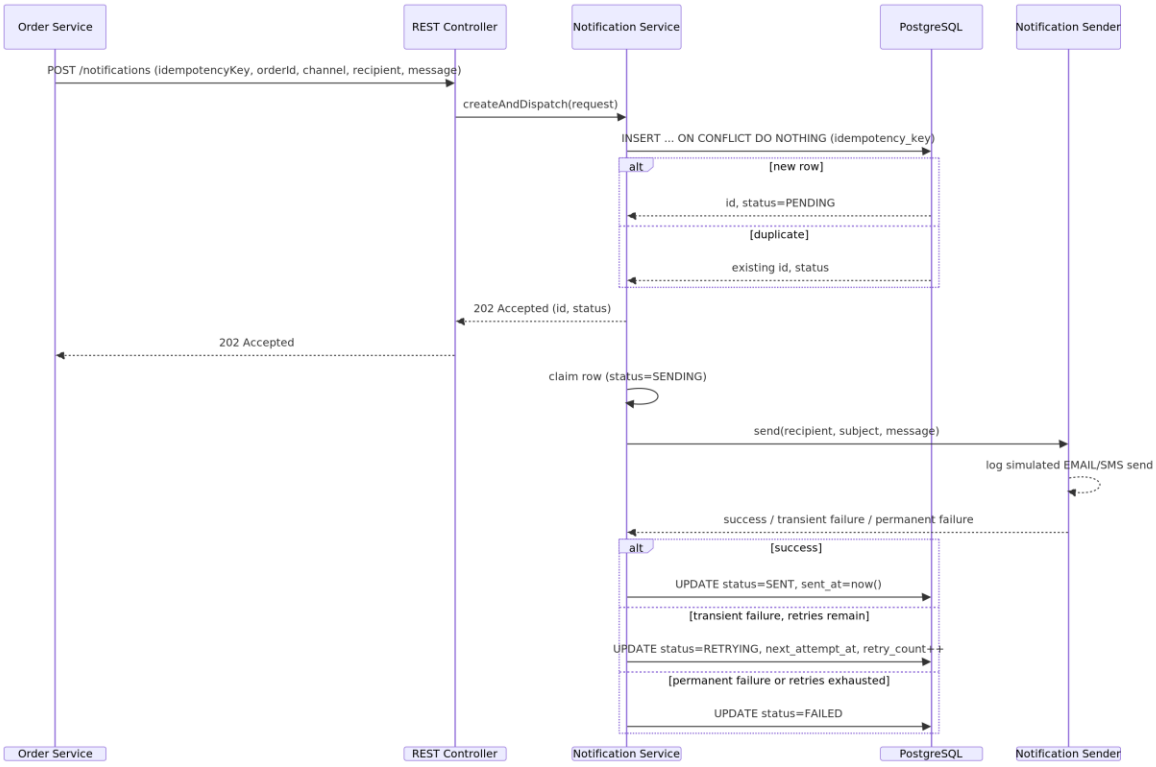

Figure 4 — Request lifecycle from POST to SENT/RETRYING/FAILED.

9. Retry & Backoff Design

- Strategy: exponential backoff with jitter. delay = min(base * 2^retryCount, cap) + jitter. Suggested defaults: base=30s, cap=1h, maxRetries=5, jitter=±20%.
- Why jitter: during a provider outage, many notifications fail at once; without jitter they retry in lockstep and hammer the provider the instant it recovers (thundering herd). Jitter spreads the retries out.
- Transient vs. permanent errors: transient (timeout, 5xx, 429) → retry. Permanent (invalid recipient, 400) → straight to FAILED; retrying is pointless and wasteful.
- Compute delay (with its random jitter component) in application code, not SQL, keeping the backoff policy in one testable place.

10. Security

- Endpoints require a shared X-API-KEY header validated via a filter/interceptor — sufficient for inter-service trust at this phase and effectively free to implement now.
- Upgrade path: OAuth2 client-credentials or mTLS once an API Gateway is introduced; the filter is designed to be swapped, not rewritten.
- Bean Validation (@Valid) on all inbound DTOs prevents malformed payloads; JPA/Hibernate parameterized queries prevent SQL injection by construction.
- PII handling: recipient (email/phone) is masked in logs (e.g., j***@example.com) — never log full message body content beyond what's operationally necessary.
- Least privilege: the service's DB user should have SELECT/INSERT/UPDATE only — no DROP/DELETE/schema-alter privileges, since no flow expects data deletion.
- Secrets (DB credentials, future provider API keys) come from environment variables or a secrets manager — never hardcoded in application.properties.

11. Logging & Observability

- Structured logs (JSON encoder in non-local profiles) with a correlation ID — the idempotencyKey/orderId — on every log line, so a single notification's journey is traceable end-to-end across Order Service → Notification Service.
- Key log points: request received → validation passed → dispatch attempted → outcome → status updated → retry scheduled (if any) → retry exhausted.
- Log levels: INFO for lifecycle transitions and retry attempts; WARN for retry-limit reached and validation rejections; ERROR for unexpected exceptions and sender failures; DEBUG for full payload dumps (disabled in production).
- Metrics to expose (Spring Boot Actuator /health, /info, /metrics): counts by status, send success/failure rate, retry rate, dead-letter (FAILED) count, delivery latency (sentAt − createdAt).
- Alerts: rising FAILED count, a growing backlog of due-but-unprocessed rows (worker stalled), frequent reaper activity (workers crashing repeatedly, Phase 2).

12. Testing Strategy

Layer	Focus	Tooling
Unit — Service	Every state transition, retry-limit logic, idempotency short-circuit	JUnit 5, Mockito
Unit — Validation	Bean Validation constraints and the channel/recipient cross-field rule	JUnit 5
Unit — Mapper	Entity <-> DTO field mapping completeness	JUnit 5
Integration — Repository	Derived queries and DB constraints against a real Postgres instance	@DataJpaTest + Testcontainers
Integration — Controller	Full HTTP round-trip: status codes, JSON shape, error responses	@SpringBootTest + MockMvc
Contract	Request/response JSON shape stays backward-compatible for Order Service	Postman/Newman or REST-assured
End-to-end	Simulated Order Service call against a Dockerized Postgres	Testcontainers + RestAssured
Coverage target	≥80% line coverage on the service package; 100% of state-machine branches	JaCoCo, enforced in CI

13. Work Breakdown (5-Developer Parallel Tracks)

Step 0 (whole team, ~30 min): agree on the Notification entity fields, Request/Response DTOs, and enum values — the shared contract everyone codes against. Do this before splitting.

Track	Owner	Scope
1 — API & Controller	Developer 1	REST controllers (all 7 endpoints), request validation, global exception handling, OpenAPI/Swagger docs, API key security filter.
2 — Domain & Persistence	Developer 2	Notification (+ NotificationAttempt) entity, JPA repository with derived/specification queries, Flyway migration scripts, enum definitions, datasource configuration.
3 — Sending Engine	Developer 3	NotificationSender interface + simulated EmailNotificationSender/SmsNotificationSender, sender factory, core orchestration (validate → persist → send → update status).
4 — Retry & Scheduling	Developer 4	Phase-1 @Scheduled retry poller, failure-simulation config, transition logic to FAILED, manual retry endpoint support; own the Phase-2 SKIP LOCKED design as a follow-up.
5 — Integration, Cross-Cutting & QA	Developer 5	API contract handoff to Order Service, logging strategy, idempotency implementation, test suite across all layers, Postman collection, Docker setup.

Dependency note: Tracks 1, 3, and 4 all depend on Track 2's entity/DTOs existing — this is why Step 0 is a quick group agreement, not a solo task.

Suggested sprint view:

Day	Focus
1	Step 0 contract agreement → Track 2 scaffolds entity/DB → others build against interfaces/stubs
2–3	Tracks 1, 3, 4 build core logic in parallel
4	Integration — wire controller → service → sender → retry job together
5	Track 5 finalizes tests, docs, Docker, and hands off the API contract to Order Service

Detailed backlog (from the single-developer estimate in the source HLD, ~11.75 dev-days total, distributed into the tracks above): resolve the Spring Boot 4.1.0 confirmation and add missing dependencies (Track 2, 0.5d); package structure (Track 2, 0.25d); entity/enums/migration (Track 2, 0.5d); repository (Track 2, 0.5d); DTOs + validation (Track 1, 1d); mapper (Track 2, 0.5d); sender SPI + implementations (Track 3, 1d); service orchestration + state machine + retry logic (Track 3/4, 1.5d); controller + all endpoints (Track 1, 1d); global exception handler (Track 1, 0.5d); retry scheduler (Track 4, 0.5d); structured logging (Track 5, 0.5d); unit tests (all tracks, 1.5d); integration tests (Track 5, 1.5d); OpenAPI docs (Track 1, 0.5d); Dockerfile/compose (Track 5, 0.5d); contract coordination with Order Service (Track 5, 0.5d); README + HLD sign-off (Track 5, 0.5d).

14. Git Branch Strategy & Pull Request Checklist

Branching (GitFlow-lite, consistent across all three sibling repos):

```
main          - always production-deployable
  +-- develop        - integration branch
     +-- feature/notification-crud-api
     +-- feature/notification-retry-mechanism
     +-- feature/notification-scheduler
     +-- feature/notification-validation
     +-- bugfix/notification-<short-desc>
     +-- hotfix/notification-<short-desc>  (branched from main directly)
```

Each repo currently has only a master branch, so develop needs to be created before feature work starts. Commit messages follow Conventional Commits (feat:, fix:, docs:, test:, refactor:).

Pull Request checklist:

- Code compiles cleanly against the confirmed Spring Boot version; no new warnings.
- All new endpoints documented in OpenAPI/Swagger.
- Bean Validation applied to every request DTO field.
- No JPA entity is ever returned directly from a controller method.
- State transitions match §6 exactly — no undocumented transitions.
- retryCount is never client-settable.
- DB constraints match entity-level validation — no drift between Java and SQL rules.
- Migration script added under db/migration (Flyway, versioned, never edited after merge).
- Unit and integration tests added/updated; coverage threshold met.
- No secrets or provider API keys committed.
- PII confirmed masked in all new log statements.
- Correlation ID propagated through any new logging.
- Reviewed by at least one other engineer plus Tech Lead sign-off before merge to develop.

15. Future Enhancements

- Kafka integration — @KafkaListener consuming an order-events topic, using the event ID as the idempotency key (§5.4).
- Real email/SMS providers — SES/SendGrid and Twilio/MSG91 implementations of NotificationSender, with provider error codes mapped to transient/permanent classification (§9).
- Phase-2 retry engine — SELECT ... FOR UPDATE SKIP LOCKED claim queue + stuck-SENDING reaper (§5.3), once horizontal scaling is needed.
- Push notifications — a third NotificationChannel value and PushNotificationSender; no core changes required.
- Notification templates — a templating layer so callers pass a template ID and parameters instead of a fully-rendered message.
- User notification preferences — opt-in/opt-out and channel preference store.
- Delivery-receipt webhooks — providers call back with final delivery confirmation, introducing a DELIVERED status beyond SENT.
- Multi-channel fan-out — a single logical event triggering Email + SMS + Push via an internal orchestrator.

16. Assumptions & Open Decisions

Assumptions carried into this design:

- Order Service calls Notification Service synchronously via REST after persisting an order and marking it confirmed; Notification Service never calls back into Order Service.
- Email/SMS sending is simulated — no real network call to a third-party provider is made in this phase.
- Each incoming request results in exactly one notification row per channel, per event; if both Email and SMS are required, the caller sends two requests.
- Order Service is the source of truth for orderId validity; Notification Service trusts the payload without a synchronous validation callback.
- PostgreSQL is the single system of record for notification history; no separate audit store beyond notification_attempts.
- Authentication between services beyond the API key (OAuth2, mTLS) is deferred to a later hardening phase; this document notes where the hook goes (§10).

Open decisions to lock before implementation (carried from the BRD, still unresolved):

- 1. Confirm Spring Boot 4.1.0 is intentional (§1.4) — blocks all other work if wrong.
- 2. idempotencyKey required, or server-generated if absent? Recommendation: require it for all callers — retrofitting dedup later is painful.
- 3. Worker deployment model for Phase 2 — start with a single in-process poller; the SKIP LOCKED claim query already supports scaling to N workers with zero code change when needed.
- 4. Confirm retry/backoff values (maxRetries, base, cap, jitter) as configuration, not hardcoded constants.
- 5. Reaper timeout (Phase 2) relative to the slowest realistic send.
- 6. Transient-vs-permanent error taxonomy — which concrete provider error codes map to which, once a real provider is chosen.

17. Conclusion

This consolidated document resolves four independently-written design artifacts for the Notification Service into one implementation-ready specification. Where the sources disagreed, the more robust or better-evidenced option was adopted explicitly (§0) rather than either picked arbitrarily or left unresolved — most notably, closing HLD's idempotency gap, upgrading its 3-state lifecycle to the crash-safe 5-state model, and phasing in BRD's production-grade Postgres-queue retry engine rather than deferring it indefinitely or shipping it prematurely.

The result keeps the parts every source agreed on (layered architecture, PostgreSQL, REST-now/Kafka-later, a swappable sender abstraction) untouched, while giving the team a single backlog (§13), a single schema (§7), and a single API contract (§8) to build against — with every deviation from any one source document traceable back to a stated reason, ready for Tech Lead review and sign-off.

— End of Document —